ETS

PEMROGRAMAN OTA BERBASIS BLYNK



Dosen Pengampu:

Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:

Arif Nur Cahyo (23081010192)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL " VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

DAFTAR ISI i
DAFTAR GAMBAR ii
BAB I KONFIGURASI DAN KODE PROGRAM 1
1.1 Konsep OTA dan Perbedaan dengan Praktikum Sebelumnya 1
1.2 Konfigurasi OTA dan Datastream Blynk 2
1.3 Kode Program 2
BAB II HASIL DAN PEMBAHASAN 9
2.1 Upload Awal melalui Kabel dan Kesiapan OTA 9
2.2 Kondisi Dashboard Sebelum OTA 9
2.3 Pengujian Kendali Motor dengan Firmware v1.0.0 10
2.4 Pengaturan Kecepatan melalui Slider 11
2.5 Mematikan Motor melalui Switch 13
2.6 Proses Upload Firmware secara OTA 13
2.7 Verifikasi Hasil OTA pada Dashboard 14
2.8 Pengujian Tanpa Kabel Data ke Laptop 15
2.9 Rangkuman Hasil Pengujian 16
BAB III PENUTUP 18
3.1 Kesimpulan 18
3.2 Saran 18

DAFTAR GAMBAR

Gambar 1. Serial Monitor menampilkan kesiapan OTA dan alamat IP 9
Gambar 2. Dashboard Blynk sebelum OTA dengan firmware v1.0.0 10
Gambar 3. Pengujian motor saat KIT masih terhubung kabel USB 11
Gambar 4. Kecepatan dinaikkan melalui slider menjadi PWM 206 12
Gambar 5. Kecepatan diturunkan melalui slider menjadi PWM 195 12
Gambar 6. Kondisi dashboard setelah Switch dimatikan.....13
Gambar 7. Dialog autentikasi saat upload melalui port jaringan 14
Gambar 8. Dashboard setelah OTA dengan firmware v1.0.1 15
Gambar 9. Pengujian perangkat tanpa kabel data ke laptop 16

BAB I KONFIGURASI DAN KODE PROGRAM

Praktikum ini merupakan pengembangan dari praktikum sebelumnya yang telah berhasil mengendalikan motor DC pada KIT iMCLab melalui dashboard Blynk. Jika sebelumnya setiap perubahan program harus diunggah melalui kabel USB, maka pada praktikum ini pembaruan firmware dilakukan secara Over-The-Air (OTA), yaitu dikirim melalui jaringan WiFi tanpa menyentuh perangkat keras sama sekali.

1.1 Konsep OTA dan Perbedaan dengan Praktikum Sebelumnya

OTA diwujudkan menggunakan pustaka ArduinoOTA yang berjalan berdampingan dengan Blynk. Keduanya sama-sama membutuhkan ESP32 berada pada mode station, sehingga dapat aktif bersamaan tanpa konflik: Blynk menangani kendali motor melalui internet, sedangkan ArduinoOTA menangani pengiriman firmware di dalam jaringan lokal.

Mekanismenya berlangsung dua arah. Arduino IDE terlebih dahulu mengirimkan undangan ke alamat IP ESP32, kemudian ESP32 membalas dan membuka koneksi balik ke laptop untuk menarik berkas firmware. Setelah seluruh data tersimpan pada partisi cadangan, board melakukan restart dan menjalankan program yang baru. Perbandingan dengan praktikum sebelumnya diringkas pada tabel berikut.

No	Aspek	Praktikum Sebelumnya
1	Pengiriman program	Kabel USB melalui port COM7
2	Posisi perangkat	Harus dekat dan tersambung ke laptop
3	Sumber daya KIT	Dari port USB laptop
4	Kendali motor	Dashboard Blynk

No	Aspek	Praktikum Ini (OTA)
1	Pengiriman program	Jaringan WiFi melalui port 172.31.242.170
2	Posisi perangkat	Cukup berada pada jaringan yang sama
3	Sumber daya KIT	Adaptor atau powerbank
4	Kendali motor	Dashboard Blynk, tetap berjalan

1.2 Konfigurasi OTA dan Datastream Blynk

Identitas OTA ditetapkan melalui dua konstanta, yaitu hostname imclab-arif sebagai nama perangkat yang tampil pada daftar port jaringan Arduino IDE, dan password robotika yang harus dimasukkan setiap kali upload agar perangkat tidak dapat diperbarui oleh sembarang komputer pada jaringan yang sama.

Sebagai penanda keberhasilan OTA, ditambahkan satu datastream baru yaitu V4 bertipe String yang menampilkan versi firmware pada dashboard. Dengan demikian seluruh pembuktian dapat diamati langsung dari Blynk tanpa bergantung pada Serial Monitor. Daftar lengkap virtual pin yang digunakan adalah sebagai berikut.

Pin	Nama	Tipe	Keterangan
V0	RPM	Double	Hasil pembacaan sensor encoder
V1	PWM	Integer	Nilai duty cycle yang sedang aktif
V2	Slider PWM	Integer	Masukan pengguna untuk mengatur kecepatan
V3	Switch	Integer	Tombol ON/OFF motor
V4	FW_VERSION	String	Versi firmware, penanda keberhasilan OTA

Selain itu ditambahkan konstanta DEFAULT_DUTY, yaitu nilai duty cycle yang dipakai ketika Switch dinyalakan dari kondisi mati. Konstanta inilah yang nanti diubah nilainya untuk membuktikan bahwa program benar-benar berganti setelah OTA. Fungsi motorDarurat() juga ditambahkan dan dipanggil pada callback onStart, karena board akan melakukan restart di tengah proses penulisan firmware sehingga motor tidak boleh dibiarkan berputar tanpa kendali.

1.3 Kode Program

Berikut adalah kode program lengkap imclab_Blynk_OTA.ino yang digunakan pada praktikum ini.

```
/*************************************************************
 * iMCLab + ESP32 + Blynk IoT + OTA
```

```
 * RPM Monitoring, Kontrol PWM, dan Update Firmware
 * secara Over-The-Air (ArduinoOTA)
 ********************************************************/

#define BLYNK_TEMPLATE_ID "TMPL6rGehYQam"
#define BLYNK_TEMPLATE_NAME "robot"
#define BLYNK_AUTH_TOKEN "DDFLFzKPinPkVqKgA_36G48y0Al5ibPc"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ArduinoOTA.h>

//char ssid[] = "HOTSPOT@UPNJATIM.AC.ID";
//char pass[] = "belanegara";

//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "HOTSPOT@UPNJATIM.AC.ID";
char pass[] = "belanegara";

// ============================================================
// OTA
// ============================================================

// Naikkan nomor ini setiap kali upload, supaya terlihat jelas
// di Serial Monitor bahwa firmware benar-benar sudah berganti.
#define FW_VERSION "1.0.0"

const char* OTA_HOSTNAME = "imclab-arif";
const char* OTA_PASSWORD = "robotika";

// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ============================================================
// Sensor RPM
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

// Nilai yang dipakai saat Switch Blynk dinyalakan dari kondisi mati.
// Angka inilah yang diubah untuk membuktikan OTA lewat dashboard.
const int DEFAULT_DUTY = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================
```

```
float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);

  // Kirim versi firmware ke widget Label (V4).
  // Inilah bukti OTA yang terlihat langsung di dashboard Blynk.
  Blynk.virtualWrite(V4, FW_VERSION);
}

// ============================================================
// Hitung RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);
```

```
  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Matikan motor secara paksa (dipakai sebelum OTA)
// ============================================================

void motorDarurat()
{
  dutyCycle = 0;
  motorEnable = false;

  ledcWrite(pwmChannel, 0);

  digitalWrite(motor1Pin1, LOW);
  digitalWrite(motor1Pin2, LOW);
  digitalWrite(ledPin, LOW);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

// ============================================================
// Kirim Data ke Blynk
// ============================================================

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
```

```
    if (dutyCycle == 0)
      dutyCycle = DEFAULT_DUTY;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Konfigurasi OTA
// ============================================================

void setupOTA()
{
  ArduinoOTA.setHostname(OTA_HOSTNAME);
  ArduinoOTA.setPassword(OTA_PASSWORD);

  ArduinoOTA.onStart([]()
  {
    // Motor WAJIB dimatikan dulu. Board akan restart di tengah
    // proses, jadi motor tidak boleh dibiarkan berputar.
    motorDarurat();

    Serial.println();
    Serial.println("================================");
    Serial.println("OTA dimulai, motor dimatikan");
    Serial.println("================================");
  });

  ArduinoOTA.onProgress([](unsigned int progress, unsigned int total)
  {
    Serial.printf("OTA: %u%%\r", (progress * 100) / total);
  });

  ArduinoOTA.onEnd([]()
  {
    Serial.println();
    Serial.println("OTA selesai. Board restart...");
  });

  ArduinoOTA.onError([](ota_error_t error)
  {
    Serial.printf("OTA Error [%u]: ", error);

    if (error == OTA_AUTH_ERROR)         Serial.println("Auth gagal");
    else if (error == OTA_BEGIN_ERROR)   Serial.println("Begin
gagal");
    else if (error == OTA_CONNECT_ERROR) Serial.println("Connect
gagal");
    else if (error == OTA_RECEIVE_ERROR) Serial.println("Receive
gagal");
    else if (error == OTA_END_ERROR)     Serial.println("End gagal");
  });

  ArduinoOTA.begin();

  Serial.println("--------------------------------");
  Serial.print("OTA siap. Hostname : ");
  Serial.println(OTA_HOSTNAME);
  Serial.print("Alamat IP          : ");
  Serial.println(WiFi.localIP());
  Serial.println("--------------------------------");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);
```

```
  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

  // Koneksi ke Blynk (sekaligus menyambungkan WiFi)
  Blynk.begin(auth, ssid, pass);

  // OTA dipasang SETELAH WiFi tersambung
  setupOTA();

  timer.setInterval(
    1000L,
    kirimKeBlynk);

  Serial.println("================================");
  Serial.print("iMCLab + Blynk + OTA Ready  v");
  Serial.println(FW_VERSION);
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
  Serial.println("V = lihat versi firmware & IP");
}

// ============================================================
// Loop
// ============================================================

void loop()
{
  Blynk.run();

  // Melayani permintaan upload OTA dari Arduino IDE
  ArduinoOTA.handle();

  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
```

```
    {
      int val = inputString.substring(4).toInt();

      val = constrain(
        val,
        DUTY_MIN,
        DUTY_MAX);

      dutyCycle = val;
      motorEnable = (dutyCycle > 0);

      applyMotor();

      tampilkanStatus("Set PWM");
    }

    else if (inputString.length() > 0)
    {
      char cmd = inputString.charAt(0);

      if (cmd == '1')
      {
        if (dutyCycle == 0)
          dutyCycle = DEFAULT_DUTY;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("Motor ON");
      }

      else if (cmd == '0')
      {
        dutyCycle = 0;
        motorEnable = false;

        applyMotor();

        Serial.println("Motor OFF");
      }

      else if (cmd == '+')
      {
        dutyCycle += DUTY_STEP;

        if (dutyCycle > DUTY_MAX)
          dutyCycle = DUTY_MAX;

        motorEnable = true;

        applyMotor();

        tampilkanStatus("PWM NAIK");
      }

      else if (cmd == '-')
      {
        dutyCycle -= DUTY_STEP;

        if (dutyCycle < DUTY_MIN)
          dutyCycle = DUTY_MIN;

        motorEnable = (dutyCycle > 0);

        applyMotor();

        tampilkanStatus("PWM TURUN");
      }

      else if (cmd == 'R' || cmd == 'r')
      {
        Serial.print("RPM = ");
        Serial.print(rpm, 1);
        Serial.print(" | PWM = ");
        Serial.println(dutyCycle);
      }
```

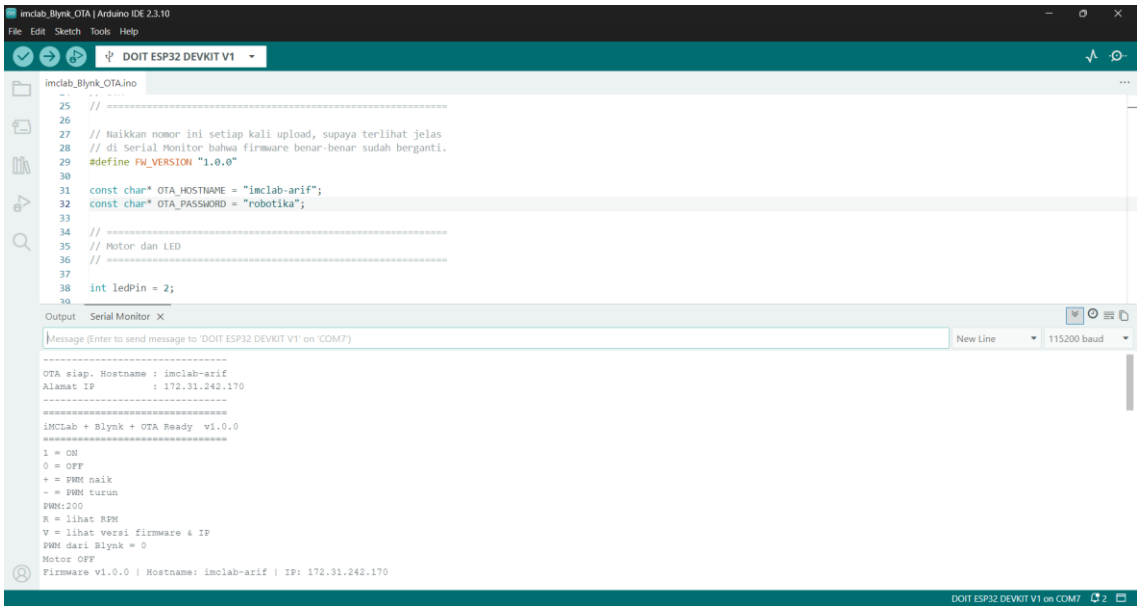
```
        else if (cmd == 'V' || cmd == 'v')
        {
          Serial.print("Firmware v");
          Serial.print(FW_VERSION);
          Serial.print(" | Hostname: ");
          Serial.print(OTA_HOSTNAME);
          Serial.print(" | IP: ");
          Serial.println(WiFi.localIP());
        }
      }

      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

BAB II HASIL DAN PEMBAHASAN

2.1 Upload Awal melalui Kabel dan Kesiapan OTA

Firmware pertama tetap harus diunggah melalui kabel USB, karena OTA baru dapat bekerja setelah ESP32 menjalankan program yang memuat ArduinoOTA. Setelah upload selesai, Serial Monitor menampilkan status kesiapan OTA.

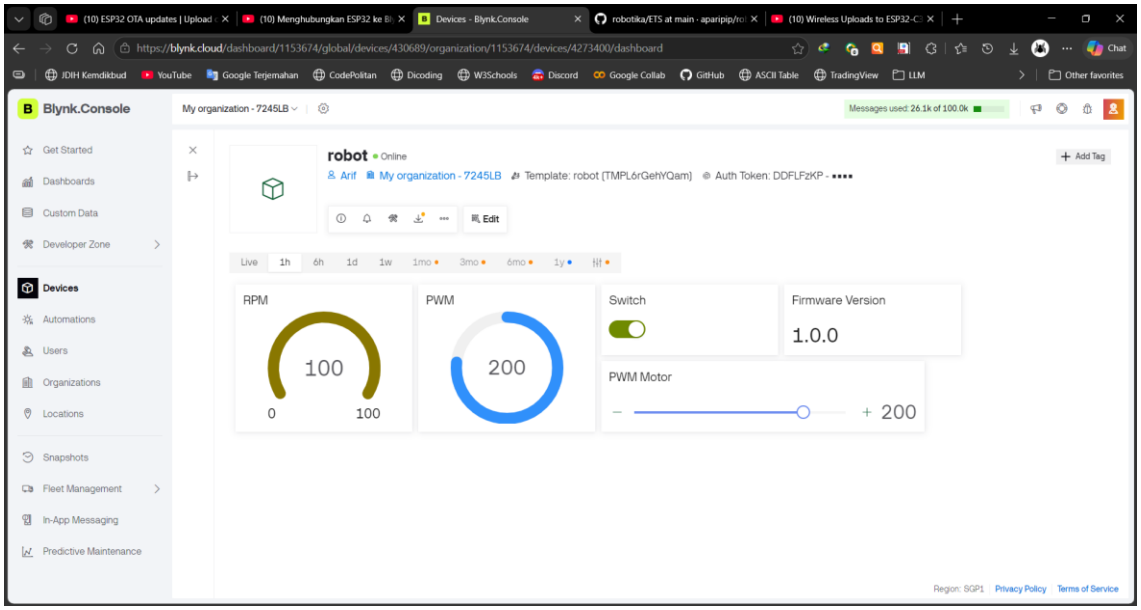


Gambar 1. Serial Monitor menampilkan kesiapan OTA dan alamat IP

Keluaran menunjukkan hostname imclab-arif dan alamat IP 172.31.242.170 yang diperoleh dari hotspot, disusul banner “iMCLab + Blynk + OTA Ready v1.0.0”. Perintah V yang dikirimkan kemudian membalas “Firmware v1.0.0 | Hostname: imclab-arif | IP: 172.31.242.170”. Alamat IP inilah yang nanti muncul sebagai port jaringan pada Arduino IDE.

2.2 Kondisi Dashboard Sebelum OTA

Setelah perangkat terhubung ke server Blynk, dashboard menampilkan kondisi awal program.

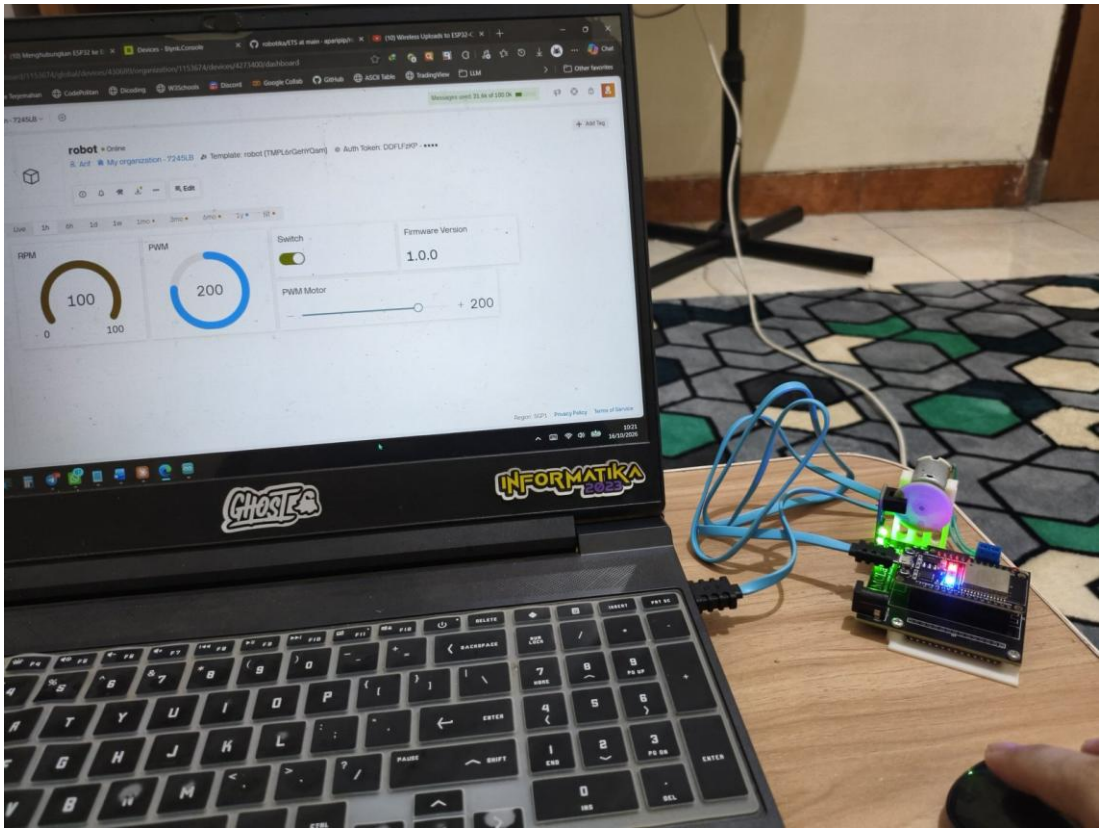


Gambar 2. Dashboard Blynk sebelum OTA dengan firmware v1.0.0

Widget Firmware Version menampilkan 1.0.0, nilai yang dikirim oleh fungsi BLYNK_CONNECTED() ke datastream V4. Widget PWM dan slider menunjukkan 200, sesuai nilai DEFAULT_DUTY pada firmware yang sedang berjalan. Kondisi ini menjadi acuan pembanding setelah OTA dilakukan.

2.3 Pengujian Kendali Motor dengan Firmware v1.0.0

Sebelum OTA dijalankan, fungsi kendali diuji terlebih dahulu untuk memastikan firmware berjalan normal.

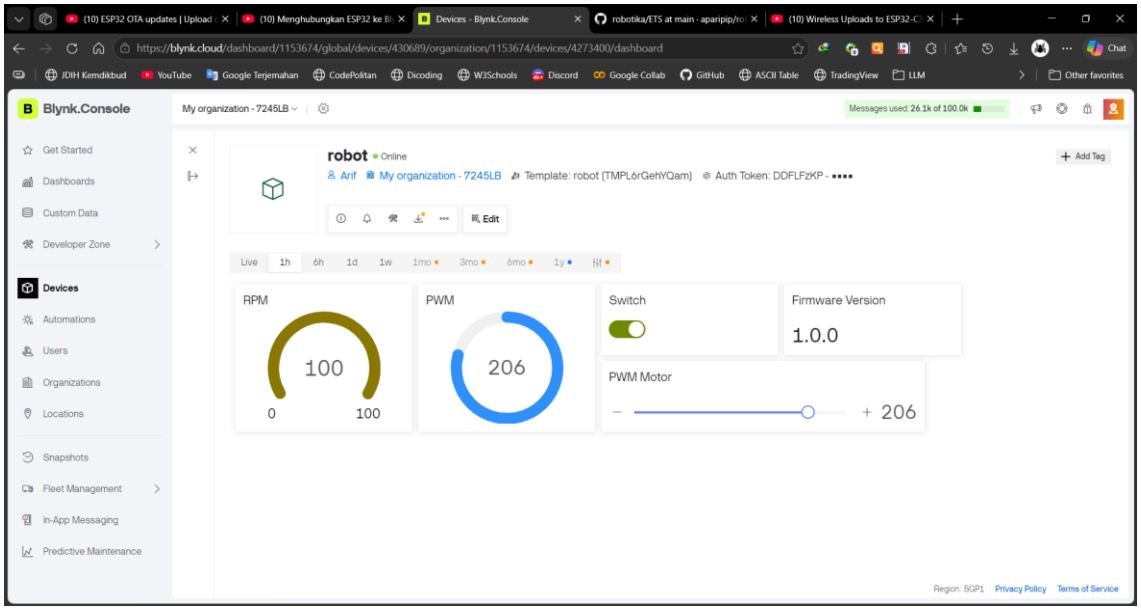


Gambar 3. Pengujian motor saat KIT masih terhubung kabel USB

Pada gambar terlihat KIT iMCLab masih tersambung ke laptop melalui kabel USB berwarna biru. Dashboard menampilkan Switch aktif, PWM 200, dan Firmware Version 1.0.0, sementara piringan motor berputar dan LED indikator biru menyala. Kondisi inilah yang menjadi pembanding langsung dengan Gambar 9 nanti.

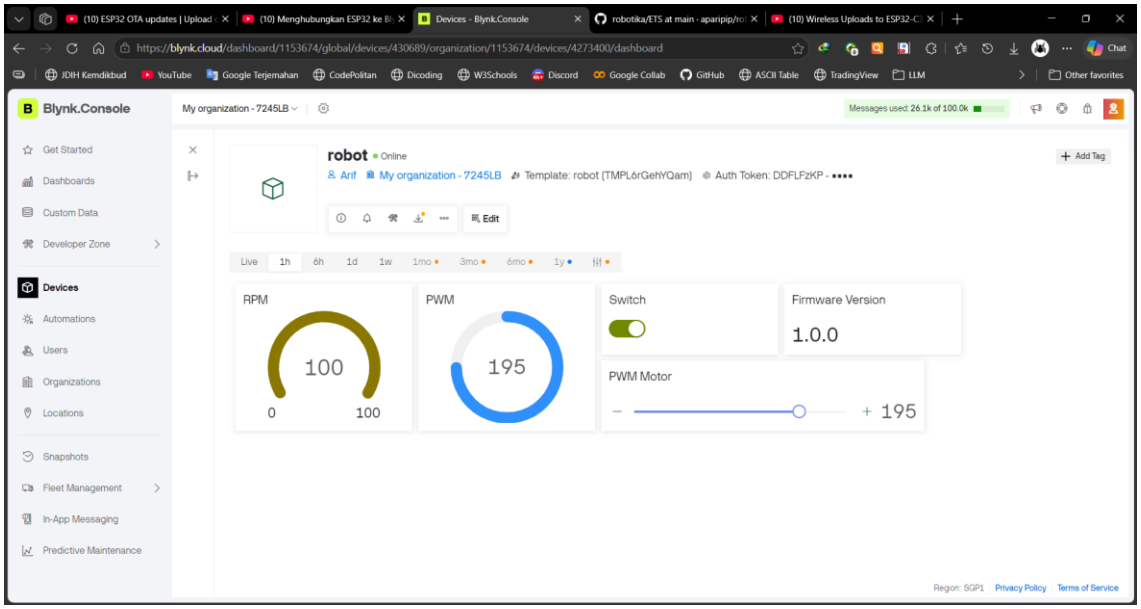
2.4 Pengaturan Kecepatan melalui Slider

Pengujian dilanjutkan dengan menggeser slider PWM Motor untuk menaikkan dan menurunkan kecepatan.



Gambar 4. Kecepatan dinaikkan melalui slider menjadi PWM 206

Slider dinaikkan hingga 206 dan widget PWM ikut berubah mengikutinya, menandakan nilai dari V2 berhasil diterima dan diteruskan ke kanal PWM oleh fungsi applyMotor().

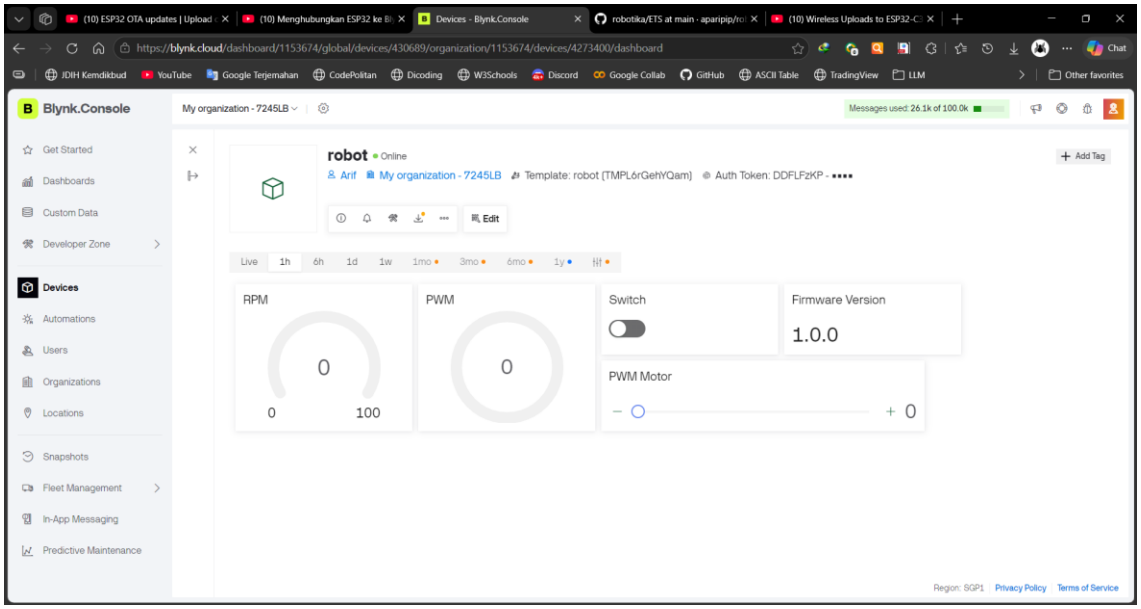


Gambar 5. Kecepatan diturunkan melalui slider menjadi PWM 195

Selanjutnya slider diturunkan menjadi 195 dan putaran motor melambat. Kedua pengujian menunjukkan kendali melalui Blynk tetap berfungsi penuh meskipun program kini juga melayani permintaan OTA pada setiap putaran loop.

2.5 Mematikan Motor melalui Switch

Switch kemudian dimatikan untuk memastikan seluruh keluaran kembali ke kondisi nol.

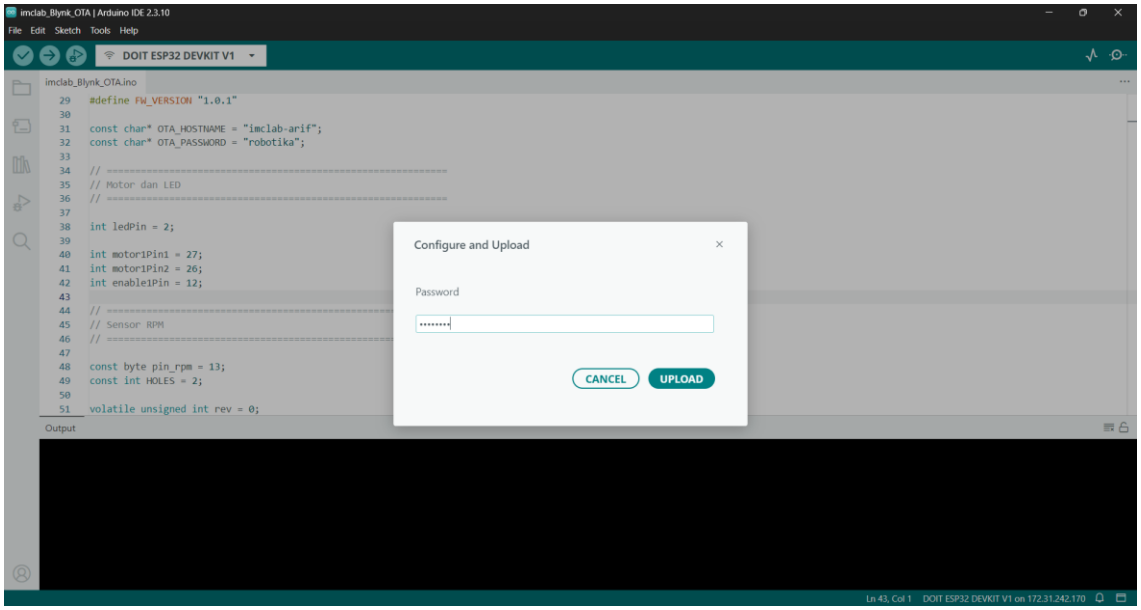


Gambar 6. Kondisi dashboard setelah Switch dimatikan

RPM, PWM, dan slider seluruhnya menunjukkan angka 0, sedangkan widget Firmware Version tetap menampilkan 1.0.0 karena versi firmware tidak terpengaruh oleh kondisi motor. Hal ini memperkuat bahwa V4 memang hanya berubah apabila program yang berjalan benar-benar berganti.

2.6 Proses Upload Firmware secara OTA

Dua baris program kemudian diubah, yaitu FW_VERSION dari 1.0.0 menjadi 1.0.1 dan DEFAULT_DUTY dari 200 menjadi 150. Port pada Arduino IDE dipindahkan dari COM7 ke port jaringan imclab-arif at 172.31.242.170, lalu proses upload dijalankan.



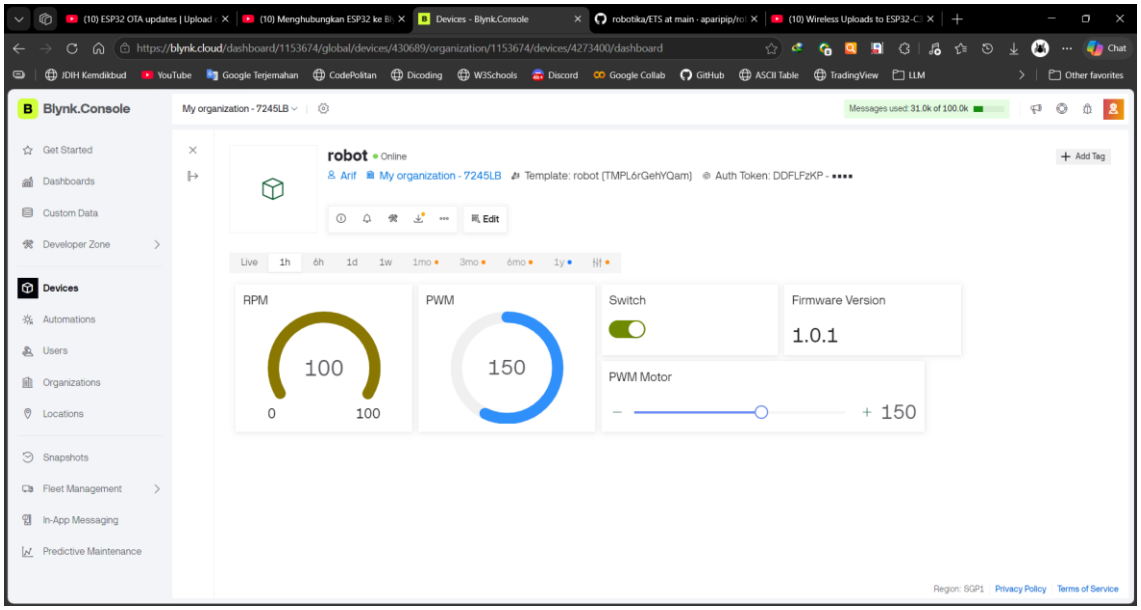
Gambar 7. Dialog autentikasi saat upload melalui port jaringan

Arduino IDE menampilkan dialog Configure and Upload yang meminta password. Dialog ini tidak pernah muncul pada upload melalui kabel, sehingga keberadaannya sendiri sudah menjadi penanda bahwa pengiriman berlangsung melalui jaringan. Password yang dimasukkan adalah robotika, sesuai konstanta OTA_PASSWORD pada program. Status bar di bagian bawah juga menampilkan "DOIT ESP32 DEVKIT V1 on 172.31.242.170", bukan COM7.

Pada percobaan pertama, upload sempat gagal dengan pesan "No response from device" meskipun tahap "Authenticating...OK" sudah terlewati. Hal ini terjadi karena ArduinoOTA mengharuskan ESP32 membuka koneksi balik ke laptop, sementara koneksi masuk tersebut tertahan oleh Windows Firewall. Setelah pengaturan jaringan disesuaikan, proses upload berhasil diselesaikan.

2.7 Verifikasi Hasil OTA pada Dashboard

Setelah upload selesai, board melakukan restart secara otomatis dan menyambung kembali ke server Blynk.

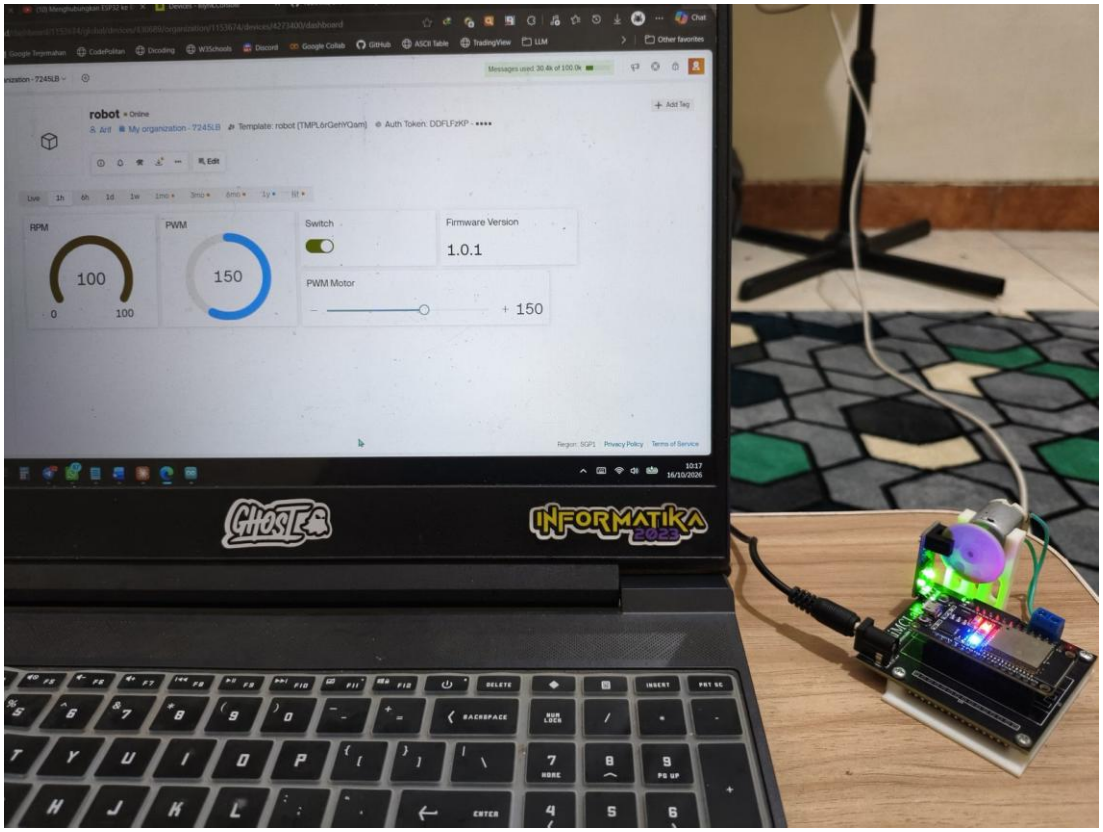


Gambar 8. Dashboard setelah OTA dengan firmware v1.0.1

Terlihat dua perubahan sekaligus. Widget Firmware Version berubah dari 1.0.0 menjadi 1.0.1, dan ketika Switch dinyalakan nilai PWM kini menunjukkan 150, bukan 200 seperti sebelumnya. Keduanya persis sesuai dengan dua baris yang diubah pada kode, sehingga terbukti firmware baru benar-benar tertanam dan dijalankan. Yang perlu digarisbawahi, seluruh perubahan ini terjadi tanpa satu pun penekanan tombol pada board maupun proses upload melalui kabel.

2.8 Pengujian Tanpa Kabel Data ke Laptop

Pengujian terakhir dilakukan untuk membuktikan bahwa perangkat sama sekali tidak bergantung pada laptop.



Gambar 9. Pengujian perangkat tanpa kabel data ke laptop

Kabel USB biru yang sebelumnya terlihat pada Gambar 3 sudah dilepas, dan KIT iMCLab kini hanya memperoleh daya dari adaptor. Meskipun demikian, dashboard tetap menampilkan Firmware Version 1.0.1 dengan PWM 150, piringan motor tetap berputar, dan LED indikator biru menyala. Inilah bukti terkuat dari praktikum ini: firmware diperbarui dan motor dikendalikan sepenuhnya melalui jaringan, sementara laptop hanya berperan sebagai pengirim program dan penampil dashboard.

2.9 Rangkuman Hasil Pengujian

No	Tahap	Hasil Pengamatan
1	Upload awal (USB)	Serial Monitor menampilkan hostname imclab-arif dan IP 172.31.242.170
2	Sebelum OTA	Firmware Version 1.0.0, PWM 200 saat Switch ON
3	Slider dinaikkan	PWM 206, putaran motor bertambah cepat
4	Slider diturunkan	PWM 195, putaran motor melambat
5	Switch OFF	RPM, PWM, dan slider menjadi 0

No	Tahap	Hasil Pengamatan
6	Upload OTA	Muncul dialog password, port berupa alamat IP
7	Sesudah OTA	Firmware Version 1.0.1, PWM 150 saat Switch ON
8	Tanpa kabel USB	Perangkat tetap Online dan motor tetap dapat dikendalikan

Seluruh tahap berjalan sesuai rancangan, dan perubahan yang diamati pada dashboard selalu konsisten dengan baris program yang dimodifikasi.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil pengujian, dapat disimpulkan beberapa hal sebagai berikut:

- Pustaka ArduinoOTA berhasil dijalankan berdampingan dengan Blynk pada satu program, karena keduanya sama-sama bekerja dengan ESP32 dalam mode station.
- Firmware pertama tetap harus diunggah melalui kabel USB, sedangkan pembaruan berikutnya dapat dilakukan melalui port jaringan pada alamat 172.31.242.170.
- Penggunaan password pada OTA terbukti berfungsi, ditandai dengan munculnya dialog autentikasi yang tidak pernah ada pada upload melalui kabel.
- Keberhasilan OTA terbukti dari dua perubahan pada dashboard, yaitu Firmware Version dari 1.0.0 menjadi 1.0.1 dan nilai PWM awal dari 200 menjadi 150.
- Kendali motor melalui Blynk tetap berfungsi normal selama OTA aktif, karena ArduinoOTA.handle() hanya dijalankan sekilas pada setiap putaran loop.
- Perangkat dapat diperbarui dan dikendalikan tanpa kabel data ke laptop, cukup dengan sumber daya dari adaptor dan koneksi ke jaringan yang sama.